

Milestone 2 Report

Smart MCQ Solver Challenge, DL / GenAI, T2 2026

Krishna (23f2001849) | July 3, 2026

Milestone 1 stayed inside classical NLP: TF-IDF vectors and cosine similarity, a lookup table that improved and then flatlined no matter what I tried. Milestone 2 forced a move into the Hugging Face ecosystem: the datasets library, WordPiece tokenization, the shapes that come out of a transformer forward pass, attention introspection, dense embeddings, zero-shot classification through NLI, and prompted generation with a small T5. I worked through the thirteen questions in order, since the later ones lean on groundwork laid by the earlier ones, especially the BERT sections.

Loading data and tokenizing

The first change from Milestone 1 was reading train.csv through `load_dataset` instead of pandas. The two look similar on the surface, but the internal representation is not: datasets uses Arrow-backed tables and lazy iteration, and `.map()` returns a new dataset rather than mutating one in place. For the combined_text question I concatenated the prompt and option A with a space inside a mapped function. Row 51 came out to 614 characters, confirmed separately with a manual length check.

The tokenizer's `vocab_size` is 30522, which looks arbitrary until you know why: BERT's WordPiece vocabulary was trained on English Wikipedia and BookCorpus, and training stops once adding another subword no longer meaningfully reduces [UNK] tokens in the corpus. It isn't rounded to a clean number because it was never meant to be one. The five special tokens, [PAD], [UNK], [CLS], [SEP], [MASK] at ids 0, 100, 101, 102, 103, sit inside that 30522, not on top of it. That distinction matters later, when I resize embeddings for fine-tuning.

Batch tokenization with `return_tensors='pt'`, `max_length=128`, padding and truncation, turned the full 2000-row prompt column into an `input_ids` tensor of shape (2000, 128). Both numbers are just the inputs I gave it: 2000 rows, 128 forced length. That was the point tokenization stopped feeling like “converting text” and started feeling like “producing a rectangular integer tensor with a mask attached.”

Inside the model

Question 5 was arithmetic before I loaded anything: BERT-base has hidden size 768 and 12 attention heads, so each head gets $768 / 12 = 64$ dimensions. That number isn't a config setting to look up, it's a forced consequence of the other two.

For the row-0 forward pass, in eval mode with dropout off, `last_hidden_state` came back as (1, 31, 768): one sequence, 31 tokens including [CLS] and [SEP], 768-dimensional representation per token. The [CLS] vector at index 0 is what a classifier head would normally pool from.

Reading an actual attention matrix (question 8) taught me more than the shape questions did. `outputs.attentions` is a tuple of 12 tensors, one per layer, each shaped (batch, heads, query, key). On the string “Light-ion fusion is a technique.”, which tokenizes into ten pieces because BERT-base-uncased splits on hyphens, I read the weight from [CLS] to “fusion” in the last layer's first head. The number alone doesn't mean much. It's one entry in a softmax row, and you'd need to compare it against the rest of that row, or average across heads, before saying anything about what the model is “looking at.” I did the exercise, but I'm not reading more into one number than it can support.

TF-IDF versus MiniLM

This is the comparison that actually connects back to Milestone 1. The M1 baseline scored MAP@3 around 0.35 to 0.40 depending on preprocessing. I built the same ranking logic with sentence-transformers/all-MiniLM-L6-v2: encoded all 2000 prompts and all five option columns into (2000, 384) matrices, ranked options by cosine similarity, kept the top three. Slower than TF-IDF, since MiniLM has to run the encoder for every string, but the whole pass finishes in about a minute on GPU.

What matters is not just the MiniLM MAP@3 itself but how many rows TF-IDF misses entirely that MiniLM catches. The prompts here are paraphrastic, mixed STEM and philosophy, and the correct option is often a synonym-swapped rewording rather than a lexical match. TF-IDF has no way to connect “supersymmetric partner Hamiltonians share eigenstates with equal energy” to a differently worded restatement of the same claim. MiniLM places both in the same neighborhood of a 384-dimensional space. The exact numbers are in the notebook, not reproduced here from memory.

Zero-shot classification and FLAN-T5 generation

The zero-shot pipeline (bart-large-mnli) formats each candidate label into an entailment template and reads the entailment logit. With multi_label=False the scores across candidates are softmax-normalized and sum to 1.0; with multi_label=True each candidate gets its own sigmoid, so the three scores are independent and can sum anywhere from 0 to 3. The gap between those two sums isn't deep, it just confirms the two modes are different objects, and it's a reminder that changing multi_label without touching downstream ranking logic will silently shift your calibration. On the row-1 supersymmetry question, the top-ranked option was the one that literally repeated “same energy” from the setup. That's text overlap on entailment features, not physics reasoning, and it's roughly what to expect from zero-shot NLI on a technical dataset.

FLAN-T5-small (question 13) was more of a demonstration than an experiment. The pipeline's text2text-generation alias no longer exists in transformers 5.x on the Kaggle image I used, undocumented and mildly frustrating to track down. The fix was to drive AutoModelForSeq2SeqLM directly with greedy decoding (do_sample=False, num_beams=1), the same deterministic mode the removed pipeline used anyway. Whether an 80M-parameter model gets the letter right on a Heidegger question is beside the point. The exercise was learning to prompt and read the output, not trusting it.

What tripped me up

Four things worth keeping around for Milestone 3. vocab_size and len(tokenizer) agree now but will diverge the moment I add tokens with add_tokens, and I'll need the second one when resizing embeddings. Forgetting model.eval() before the attention run made question 7's answer nondeterministic in a way the rounding almost hid. The text2text-generation pipeline alias being gone in transformers 5.x cost real debugging time for no good reason. And outputs.attentions is a tuple of 12 tensors, not one tensor with 12 layers along an axis, so attentions[-1, 0, 0] throws while attentions[-1][0, 0] works. I made that mistake once.

Into Milestone 3

Three things carry forward. The MiniLM ranker from question 10 is now the baseline to beat, not TF-IDF; if a fine-tuned model doesn't clear it by a comfortable margin, the fine-tuning went badly. The pipeline abstraction is convenient but brittle, so Milestone 3 uses AutoTokenizer and AutoModelForMultipleChoice

directly, the same pattern the FLAN-T5 workaround already forced. And attention introspection is worth repeating after fine-tuning, to see whether the patterns shift toward option-versus-prompt alignment instead of whatever the untrained head happened to attend to.

Files in this branch

notebooks/milestone-2.ipynb : the notebook answering all 13 questions in order, ANSWERS dictionary populated at the end.

src/m2_pipelines.py : extracted helper functions so Milestone 3 can import the MiniLM ranker and MAP@3 scorer.

reports/milestone-2-report.pdf : this document.

milestone-2-answers.json : machine-readable dump of the numerical answers.